

```

<!DOCTYPE html><html lang="en"><head> <meta charset="UTF-8"> <meta name="viewport"
content="width=device-width, initial-scale=1.0"> <title>SINGULARITY GRID: DIRECT LOCAL
LINK</title> <link
href="https://fonts.googleapis.com/css2?family=Plus+Jakarta+Sans:wght@300;400;600;800&di
splay=swap" rel="stylesheet"> <style> :root { --accent-x: #0284c7; /* Elegant Slate Blue /
--accent-o: #e11d48; / Warm Crimson / --accent-neutral: #d97706; / Ochre / --accent-bt:
#2563eb; / Bluetooth Blue / --bg-canvas: #f8fafc; --bg-card: rgba(255, 255, 255, 0.95);
--border-color: #e2e8f0; --text-main: #0f172a; --text-muted: #64748b; --cell-size: 100px; } * {
box-sizing: border-box; margin: 0; padding: 0; user-select: none; -webkit-user-select: none; }
body { background-color: var(--bg-canvas); color: var(--text-main); font-family: 'Plus Jakarta
Sans', sans-serif; min-height: 100vh; display: flex; flex-direction: column; align-items: center;
justify-content: flex-start; position: relative; overflow-y: auto; / Scrollable design */ padding: 30px
15px; } body::before { content: ""; position: fixed; top: 0; left: 0; width: 100vw; height: 100vh;
opacity: 0.03; pointer-events: none; background-image: radial-gradient(#000 1px, transparent
0), radial-gradient(#000 1px, transparent 0); background-size: 16px 16px; background-position:
0 0, 8px 8px; z-index: 1; } canvas#particleCanvas { position: fixed; top: 0; left: 0; width: 100vw;
height: 100vh; z-index: 1; pointer-events: none; } .app-container { position: relative; z-index: 2;
width: 100%; max-width: 960px; display: flex; flex-direction: column; gap: 25px; } header {
text-align: center; padding-bottom: 5px; } h1 { font-size: 2.3rem; font-weight: 800; color:
var(--text-main); letter-spacing: -0.04em; margin-bottom: 4px; } .subtitle { font-size: 0.8rem;
color: var(--text-muted); letter-spacing: 2px; text-transform: uppercase; font-weight: 600; }
.main-grid { display: grid; grid-template-columns: 1fr 320px; gap: 25px; } @media (max-width:
900px) { .main-grid { grid-template-columns: 1fr; } } .panel { background: var(--bg-card); border:
1px solid var(--border-color); border-radius: 20px; padding: 24px; box-shadow: 0 4px 20px
rgba(0, 0, 0, 0.02), inset 0 2px 4px rgba(255, 255, 255, 0.8); transition: all 0.3s cubic-bezier(0.4,
0, 0.2, 1); } .panel:hover { box-shadow: 0 10px 30px rgba(0, 0, 0, 0.04); border-color: #cbd5e1; }
.panel-title { font-size: 0.95rem; font-weight: 700; letter-spacing: 0.5px; margin-bottom: 20px;
text-transform: uppercase; color: var(--text-main); display: flex; justify-content: space-between;
align-items: center; border-bottom: 1px solid var(--border-color); padding-bottom: 10px; }
/* Single Scoreboard vs Grid */ .scoreboard-single { display: grid; grid-template-columns: 1fr
auto 1fr; align-items: center; gap: 12px; margin-bottom: 20px; } .score-divider { font-size:
0.75rem; font-weight: 800; color: var(--text-muted); text-transform: uppercase; letter-spacing:
1px; padding: 0 6px; } .score-card { background: #f1f5f9; border: 1px solid transparent;
border-radius: 12px; padding: 14px; text-align: center; transition: all 0.3s cubic-bezier(0.4, 0, 0.2,
1); } .score-val { font-size: 1.8rem; font-weight: 800; } .score-val.player { color: var(--accent-x); }
.score-val.ai { color: var(--accent-o); } .score-label { font-size: 0.65rem; text-transform:
uppercase; color: var(--text-muted); font-weight: 700; margin-top: 4px; white-space: nowrap;
overflow: hidden; text-overflow: ellipsis; }
/* Scary Shaking Keyframes / @keyframes scaryShake { 0% { transform: translate(0, 0)
rotate(0deg); } 10% { transform: translate(-2px, 2px) rotate(-1.5deg); } 20% { transform:
translate(2px, -1px) rotate(1.5deg); } 30% { transform: translate(-1px, -2px) rotate(-1deg); } 40%
{ transform: translate(2px, 2px) rotate(1deg); } 50% { transform: translate(-2px, 1px)
rotate(-1.5deg); } 60% { transform: translate(1px, -1px) rotate(1deg); } 70% { transform:

```

```

translate(-2px, -2px) rotate(-1deg); } 80% { transform: translate(2px, 2px) rotate(1.5deg); } 90% {
transform: translate(-1px, 1px) rotate(-1deg); } 100% { transform: translate(0, 0) rotate(0deg); } }
.scary-shake { animation: scaryShake 0.4s infinite; background: #ffe4e6 !important; / Soft rose
warning block */ border-color: rgba(225, 29, 72, 0.3) !important; box-shadow: 0 0 15px rgba(225,
29, 72, 0.1); } .scary-shake .score-val { color: var(--accent-o) !important; }
.menu-group { display: flex; flex-direction: column; gap: 10px; } .btn { background: #ffffff; border:
1px solid #e2e8f0; color: var(--text-main); padding: 12px 18px; border-radius: 12px; font-family:
inherit; font-weight: 600; text-transform: uppercase; font-size: 0.75rem; letter-spacing: 0.5px;
cursor: pointer; transition: all 0.2s ease; box-shadow: 0 1px 3px rgba(0,0,0,0.02); display: flex;
align-items: center; justify-content: center; gap: 8px; } .btn:hover { background: #f8fafc;
border-color: #cbd5e1; transform: translateY(-1px); } .btn.active { background: var(--text-main);
border-color: var(--text-main); color: #ffffff; box-shadow: 0 4px 12px rgba(15, 23, 42, 0.15); }
.btn.bluetooth-btn { background: #eff6ff; border-color: #bfdbfe; color: var(--accent-bt); }
.btn.bluetooth-btn:hover { background: #dbeafe; } .btn.bluetooth-btn.active { background:
var(--accent-bt); border-color: var(--accent-bt); color: #ffffff; } .game-arena { display: flex;
flex-direction: column; align-items: center; justify-content: center; } .status-container { width:
100%; text-align: center; margin-bottom: 20px; min-height: 45px; } .status-display { font-size:
0.95rem; font-weight: 700; text-transform: uppercase; letter-spacing: 1px; color: var(--text-main);
} .reset-progress-bar { width: 0%; height: 3px; background: var(--accent-x); margin: 6px auto 0;
border-radius: 2px; transition: width 2.5s linear; } .board { display: grid; grid-template-columns:
repeat(3, 1fr); grid-template-rows: repeat(3, 1fr); gap: 14px; width: 320px; height: 320px;
position: relative; } @media (max-width: 400px) { .board { width: 270px; height: 270px; gap:
10px; } } .cell { background: #ffffff; border: 1px solid var(--border-color); border-radius: 14px;
cursor: pointer; display: flex; align-items: center; justify-content: center; font-size: 2.8rem;
font-weight: 800; position: relative; transition: all 0.2s ease; box-shadow: 0 2px 8px
rgba(0,0,0,0.01); } .cell:hover { background: #fafafa; border-color: #cbd5e1; } .cell.x::after {
content: 'X'; color: var(--accent-x); animation: cellEntrance 0.3s cubic-bezier(0.175, 0.885, 0.32,
1.275) forwards; } .cell.o::after { content: 'O'; color: var(--accent-o); animation: cellEntrance 0.3s
cubic-bezier(0.175, 0.885, 0.32, 1.275) forwards; } @keyframes cellEntrance { 0% { transform:
scale(0.4); opacity: 0; } 100% { transform: scale(1); opacity: 1; } } .winning-vector { position:
absolute; background: linear-gradient(90deg, var(--accent-x), var(--accent-o)); z-index: 5;
border-radius: 4px; box-shadow: 0 2px 10px rgba(0,0,0,0.1); transition: all 0.4s cubic-bezier(0.4,
0, 0.2, 1); pointer-events: none; }
/* P2P Bluetooth Matchmaking UI Panels */ .p2p-panel { display: none; flex-direction: column;
gap: 15px; background: #f8fafc; border: 1px dashed #cbd5e1; border-radius: 14px; padding:
16px; margin-top: 15px; } .p2p-panel.visible { display: flex; } .p2p-title { font-size: 0.8rem;
font-weight: 800; text-transform: uppercase; color: var(--accent-bt); display: flex; align-items:
center; gap: 6px; } .p2p-textarea { width: 100%; height: 70px; font-family: monospace; font-size:
0.7rem; padding: 8px; border: 1px solid #cbd5e1; border-radius: 8px; resize: none; background:
#fff; } .p2p-buttons { display: flex; gap: 8px; } .p2p-buttons .btn { flex: 1; padding: 8px; font-size:
0.7rem; }
/* Reaction Overlay System on Play Board Canvas */ .reaction-bar-container { display: none;
width: 100%; justify-content: center; margin-top: 15px; } .reaction-bar-container.visible { display:

```

```

flex; } .reaction-bar { display: flex; align-items: center; justify-content: center; gap: 12px;
background: rgba(255, 255, 255, 0.95); border: 1px solid var(--border-color); border-radius:
40px; padding: 6px 16px; box-shadow: 0 4px 15px rgba(0, 0, 0, 0.05); } .reaction-btn {
background: none; border: none; font-size: 1.5rem; cursor: pointer; transition: transform 0.25s
cubic-bezier(0.175, 0.885, 0.32, 1.275); } .reaction-btn:hover { transform: scale(1.35)
translateY(-2px); } .reaction-btn:active { transform: scale(0.95); } .reaction-bubble { position:
absolute; font-size: 3.5rem; pointer-events: none; z-index: 100; animation: popUpFloat 1.6s
cubic-bezier(0.25, 1, 0.5, 1) forwards; } @keyframes popUpFloat { 0% { opacity: 0; transform:
translate(-50%, -50%) scale(0.3) rotate(-15deg); } 15% { opacity: 1; transform: translate(-50%,
-80%) scale(1.5) rotate(10deg); } 30% { transform: translate(-50%, -100%) scale(1) rotate(0deg);
} 100% { opacity: 0; transform: translate(-50%, -200%) scale(0.7) rotate(-5deg); } }
/* Pathway Roadmap */ .story-track-panel { grid-column: 1 / -1; position: relative; width: 100%;
height: 290px; background: var(--bg-card); border: 1px solid var(--border-color); border-radius:
20px; padding: 24px; overflow-x: auto; overflow-y: hidden; display: flex; flex-direction: column;
justify-content: space-between; } .roadmap-viewport { position: relative; width: 2100px; height:
140px; margin-top: 10px; } .roadmap-svg { position: absolute; top: 0; left: 0; width: 100%; height:
100%; z-index: 2; } .road-path { fill: none; stroke: #f1f5f9; stroke-width: 8; stroke-linecap: round;
stroke-linejoin: round; } .road-path-unlocked { fill: none; stroke: url(#vectorGradient);
stroke-width: 6; stroke-linecap: round; stroke-linejoin: round; stroke-dasharray: 2500;
stroke-dashoffset: 2500; transition: stroke-dashoffset 1.5s cubic-bezier(0.4, 0, 0.2, 1); }
.chapter-node { position: absolute; width: 32px; height: 32px; border-radius: 50%; background:
#ffffff; border: 2px solid #cbd5e1; z-index: 3; transform: translate(-16px, -16px); cursor: pointer;
display: flex; align-items: center; justify-content: center; font-size: 11px; font-weight: 700; color:
#94a3b8; transition: all 0.3s cubic-bezier(0.175, 0.885, 0.32, 1.275); } .chapter-node.unlocked {
border-color: var(--accent-x); color: var(--accent-x); } .chapter-node.current { border-color:
var(--text-main); background: var(--text-main); color: #ffffff; transform: translate(-16px, -16px)
scale(1.25); } .chapter-node.completed { border-color: var(--accent-x); background:
var(--accent-x); color: #ffffff; } .pulse-car { position: absolute; width: 16px; height: 16px;
background: var(--text-main); border-radius: 50%; z-index: 4; transform: translate(-8px, -8px);
box-shadow: 0 0 10px rgba(0, 0, 0, 0.15); pointer-events: none; transition: left 1.2s
cubic-bezier(0.25, 1, 0.5, 1), top 1.2s cubic-bezier(0.25, 1, 0.5, 1); } .audio-system { position:
absolute; top: 30px; right: 30px; z-index: 10; } @media (max-width: 600px) { .audio-system { top:
15px; right: 15px; } } .audio-toggle-btn { background: #ffffff; border: 1px solid var(--border-color);
color: var(--text-muted); width: 44px; height: 44px; border-radius: 50%; cursor: pointer; display:
flex; align-items: center; justify-content: center; box-shadow: 0 2px 8px rgba(0,0,0,0.03);
transition: all 0.2s ease; } .audio-toggle-btn:hover { box-shadow: 0 4px 12px rgba(0,0,0,0.08);
color: var(--text-main); } </style> </head> <body> <canvas id="particleCanvas"></canvas> <div
class="audio-system"> <button class="audio-toggle-btn" id="audioToggle" title="Toggle Audio
Synthesizer"> <svg width="20" height="20" viewBox="0 0 24 24" fill="none"
stroke="currentColor" stroke-width="2"> <polygon points="11 5 6 9 2 9 2 15 6 15 11 19 11
5"></polygon> <path d="M19.07 4.93a10 10 0 0 1 0 14.14M15.54 8.46a5 5 0 0 1 0 7.07"
id="audioWaves" style="display: none;"></path> </svg> </button> </div> <div
class="app-container"> <header> <h1>Singularity Grid</h1> <div class="subtitle">Direct Link

```

```

Loop</div> </header> <div class="main-grid"> <div class="panel game-panel"> <div
class="panel-title"> <span>Simulation Arena</span> <span id="activeModeText" style="color:
var(--accent-x); font-size: 0.8rem; font-weight: 700;">Local Link</span> </div> <div
class="game-arena"> <div class="status-container"> <div class="status-display"
id="systemStatus">ESTABLISHING INTERFACE...</div> <div class="reset-progress-bar"
id="resetProgressBar"></div> </div> <div class="board" id="gameBoard"> <div class="cell"
data-idx="0"></div> <div class="cell" data-idx="1"></div> <div class="cell" data-idx="2"></div>
<div class="cell" data-idx="3"></div> <div class="cell" data-idx="4"></div> <div class="cell"
data-idx="5"></div> <div class="cell" data-idx="6"></div> <div class="cell" data-idx="7"></div>
<div class="cell" data-idx="8"></div> <div id="winningVector" class="winning-vector"
style="display: none;"></div> </div>
<!-- Emoji Reaction Tray for Connected Link Modes --> <div class="reaction-bar-container"
id="reactionBarContainer"> <div class="reaction-bar"> <button class="reaction-btn"
data-emoji="😭">😭</button> <button class="reaction-btn" data-emoji="❤️">❤️</button>
<button class="reaction-btn" data-emoji="😄">😄</button> <button class="reaction-btn"
data-emoji="😁">😁</button> <button class="reaction-btn" data-emoji="😬">😬</button>
</div> </div> </div> </div> <div class="panel control-panel"> <div
class="panel-title">Metrics</div>
<!-- Consolidated Single Scoreboard (My vs Opponent) --> <div class="scoreboard-single"> <div
class="score-card" id="scoreCardMy"> <div class="score-val player" id="valMy">0</div> <div
class="score-label" id="labelMy">My Score</div> </div> <div class="score-divider">VS</div>
<div class="score-card" id="scoreCardOpp"> <div class="score-val ai" id="valOpp">0</div> <div
class="score-label" id="labelOpp">Opponent</div> </div> </div>
<div class="panel-title">Protocol</div> <div class="menu-group"> <button class="btn active"
id="modePvP">Local PvP</button> <button class="btn" id="modeEasy">AI Easy
Subroutine</button> <button class="btn" id="modelImpossible">AI Minimax Engine</button>
<button class="btn bluetooth-btn" id="modeP2P"> <svg width="14" height="14" viewBox="0 0
24 24" fill="none" stroke="currentColor" stroke-width="2"> <path d="M7 7 10 10 17 7 17 10 20 7
20 10"></path> </svg> Bluetooth Web Link </button>
<!-- P2P Pairing Portal --> <div class="p2p-panel" id="p2pPanel"> <div class="p2p-title"> <span
id="p2pRoleStatus">Initializing Link Protocol...</span> </div> <div id="p2pSetupSection"> <div
style="font-size: 0.7rem; color: var(--text-muted); margin-bottom: 8px;"> Choose role to sync
devices directly: </div> <div class="p2p-buttons" style="margin-bottom: 10px;"> <button
class="btn" id="btnP2PHost">Host Session (X)</button> <button class="btn"
id="btnP2PClient">Join Session (O)</button> </div> </div> <div id="p2pExchangeSection"
style="display: none;"> <p style="font-size: 0.7rem; color: var(--text-muted); margin-bottom:
5px;" id="p2pInstructions"> Generating session parameters... </p> <textarea
class="p2p-textarea" id="p2pTokenArea" readonly placeholder="Token will generate
here..."></textarea> <div class="p2p-buttons" style="margin-top: 8px;"> <button class="btn"
id="btnCopyToken">Copy Code</button> <button class="btn" id="btnProceedToken">Confirm
Code</button> </div> </div> </div>
<button class="btn" id="btnRestart">Clear Grid</button> <button class="btn" id="btnPurge"
style="border-color: rgba(225, 29, 72, 0.35); font-size: 0.75rem;">Reset Storage</button> </div>

```

```

</div> <div class="story-track-panel"> <div class="panel-title" style="margin-bottom: 4px;">
<span>Neural Pathway Roadmap</span> <span id="roadmapTitle" style="color:
var(--accent-o); font-size: 0.85rem; font-weight: 700;">Chapter 1: Genesis Matrix</span> </div>
<div class="chapter-details-box" id="chapterSummary"> Initialize local memory conduits. Defeat
peripheral logic nodes. </div> <div class="roadmap-viewport" id="roadmapViewport"> <svg
class="roadmap-svg" id="roadmapSvg"> <defs> <linearGradient id="vectorGradient" x1="0%"
y1="0%" x2="100%" y2="0%"> <stop offset="0%" stop-color="var(--accent-x)" /> <stop
offset="100%" stop-color="var(--accent-o)" /> </linearGradient> </defs> <path class="road-path"
id="roadBase" d=""></path> <path class="road-path-unlocked" id="roadProgress" d=""></path>
</svg> <div class="pulse-car" id="pulseCar"></div> </div> </div> </div> </div> <script> const
gameChapters = [ { id: 1, name: "Genesis Matrix", desc: "Awaken inside the central sandbox
network. (AI Level: Easy)", ai: "easy" }, { id: 2, name: "Conduit Breach", desc: "Forced transit
through localized security gateways. (AI Level: Easy)", ai: "easy" }, { id: 3, name: "Silicon Drift",
desc: "Infiltrate long-distance memory buses. (AI Level: Easy)", ai: "easy" }, { id: 4, name: "Clock
Cycle Divergence", desc: "Diverting security resources via synchronized logical imbalances. (AI
Level: Easy)", ai: "easy" }, { id: 5, name: "Sub-Zero Cache", desc: "Access the cryo-preserved
data storage vaults. (AI Level: Medium)", ai: "medium" }, { id: 6, name: "Array Overload", desc:
"Initiate high-frequency memory allocation sweeps. (AI Level: Medium)", ai: "medium" }, { id: 7,
name: "Trace Route Terminal", desc: "Encounter early system heuristics. (AI Level: Medium)",
ai: "medium" }, { id: 8, name: "Dynamic Leak Cascade", desc: "Overwhelm automated Garbage
Collection sweeps. (AI Level: Medium)", ai: "medium" }, { id: 9, name: "Compiler Sanctuary",
desc: "Penetrate legacy execution areas. (AI Level: Hard)", ai: "hard" }, { id: 10, name: "Memory
Map Bleed", desc: "Manipulate address translation tables. (AI Level: Hard)", ai: "hard" }, { id: 11,
name: "The Gateway Bridge", desc: "Infiltrate trans-oceanic fiber nodes. (AI Level: Hard)", ai:
"hard" }, { id: 12, name: "Algorithmic Cascade", desc: "Engage early defensive heuristic engines.
(AI Level: Hard)", ai: "hard" }, { id: 13, name: "Kernel Void", desc: "Direct interface with low-level
execution pipelines. (AI Level: Hard)", ai: "hard" }, { id: 14, name: "Optical Bus Vault", desc:
"Bypass high-density optical routing matrices. (AI Level: Hard)", ai: "hard" }, { id: 15, name:
"Vector Domain Drift", desc: "Navigate volatile, floating-point coordinates. (AI Level:
Impossible)", ai: "impossible" }, { id: 16, name: "State Space Collision", desc: "Encounter
adaptive heuristic evaluation loops. (AI Level: Impossible)", ai: "impossible" }, { id: 17, name:
"Quantum Superposition", desc: "Decollapse superposition states. (AI Level: Impossible)", ai:
"impossible" }, { id: 18, name: "The Sentient Kernel", desc: "Direct interface with the executive
routine. (AI Level: Impossible)", ai: "impossible" }, { id: 19, name: "Singularity Core Threshold",
desc: "Cross the final logic boundary. (AI Level: Impossible)", ai: "impossible" }, { id: 20, name:
"Infinite Convergence", desc: "Integrate fully with central intelligence. (AI Level: Impossible)", ai:
"impossible" } ];
let state = { board: Array(9).fill(""), currentPlayer: "X", mode: "pvp", // pvp, easy, impossible,
story, p2p scores: { x: 0, o: 0, draws: 0 }, activeChapter: 0, unlockedChapter: 0, running: true,
autoClearTimer: null,
// P2P WebRTC State p2pRole: null, // 'host' or 'client' p2pState: 'setup', // 'setup',
'waiting-for-answer', 'pasting-answer', 'connected' mySide: 'X' // Host plays X, Client plays O };
// WebRTC Direct connection variables let peerConnection = null; let dataChannel = null;

```

```

class AudioEngine { constructor() { this.ctx = null; this.droneOsc = null; this.droneGain = null;
this.filter = null; this.lfo = null; this.lfoGain = null; this.muted = true; } init() { if (this.ctx) return;
this.ctx = new (window.AudioContext || window.webkitAudioContext)(); this.buildAtmosphere(); }
buildAtmosphere() { if (!this.ctx) return; this.droneOsc = this.ctx.createOscillator();
this.droneGain = this.ctx.createGain(); this.filter = this.ctx.createBiquadFilter();
this.droneOsc.type = "sawtooth"; this.droneOsc.frequency.setValueAtTime(65.41,
this.ctx.currentTime); this.filter.type = "bandpass"; this.filter.frequency.setValueAtTime(130.81,
this.ctx.currentTime); this.filter.Q.setValueAtTime(1.5, this.ctx.currentTime);
this.droneGain.gain.setValueAtTime(0, this.ctx.currentTime); this.droneOsc.connect(this.filter);
this.filter.connect(this.droneGain); this.droneGain.connect(this.ctx.destination);
this.droneOsc.start();
this.lfo = this.ctx.createOscillator(); this.lfoGain = this.ctx.createGain();
this.lfo.frequency.setValueAtTime(0.15, this.ctx.currentTime);
this.lfoGain.gain.setValueAtTime(35, this.ctx.currentTime); this.lfo.connect(this.lfoGain);
this.lfoGain.connect(this.filter.frequency); this.lfo.start(); } triggerClick() { if (!this.ctx || this.muted)
return; this.ctx.resume(); const t = this.ctx.currentTime; const osc = this.ctx.createOscillator();
const gainNode = this.ctx.createGain(); osc.type = "sine"; osc.frequency.setValueAtTime(900, t);
osc.frequency.exponentialRampToValueAtTime(120, t + 0.12);
gainNode.gain.setValueAtTime(0.12, t); gainNode.gain.linearRampToValueAtTime(0.001, t +
0.12); osc.connect(gainNode); gainNode.connect(this.ctx.destination); osc.start(t); osc.stop(t +
0.13); } triggerConfirm() { if (!this.ctx || this.muted) return; this.ctx.resume(); const t =
this.ctx.currentTime; const osc = this.ctx.createOscillator(); const gainNode =
this.ctx.createGain(); osc.type = "triangle"; osc.frequency.setValueAtTime(329.63, t);
osc.frequency.setValueAtTime(493.88, t + 0.08); gainNode.gain.setValueAtTime(0.18, t);
gainNode.gain.linearRampToValueAtTime(0.001, t + 0.28); osc.connect(gainNode);
gainNode.connect(this.ctx.destination); osc.start(t); osc.stop(t + 0.29); } triggerFailure() { if
(!this.ctx || this.muted) return; this.ctx.resume(); const t = this.ctx.currentTime; const osc =
this.ctx.createOscillator(); const gainNode = this.ctx.createGain(); osc.type = "sawtooth";
osc.frequency.setValueAtTime(160, t); osc.frequency.linearRampToValueAtTime(70, t + 0.35);
gainNode.gain.setValueAtTime(0.14, t); gainNode.gain.linearRampToValueAtTime(0.001, t +
0.38); osc.connect(gainNode); gainNode.connect(this.ctx.destination); osc.start(t); osc.stop(t +
0.39); } setMuteState(mute) { this.muted = mute; if (!this.ctx) this.init(); if (this.ctx) {
this.ctx.resume(); const t = this.ctx.currentTime; if (mute) {
this.droneGain.gain.linearRampToValueAtTime(0, t + 0.4); } else {
this.droneGain.gain.linearRampToValueAtTime(0.04, t + 0.4); } } } } const audio = new
AudioEngine();

class ParticleEngine { constructor() { this.canvas = document.getElementById("particleCanvas");
this.ctx = this.canvas.getContext("2d"); this.particles = []; this.cursorX = 0; this.cursorY = 0;
this.init(); } init() { this.resize(); window.addEventListener("resize", () => this.resize());
window.addEventListener("mousemove", (e) => { this.cursorX = e.clientX; this.cursorY =
e.clientY; }); window.addEventListener("touchmove", (e) => { if (e.touches && e.touches[0]) {
this.cursorX = e.touches[0].clientX; this.cursorY = e.touches[0].clientY; } }); this.generate(45);
this.renderLoop(); } resize() { this.canvas.width = window.innerWidth; this.canvas.height =

```

```

window.innerHeight; } generate(count) { for (let i = 0; i < count; i++) { this.particles.push({ x:
Math.random() * this.canvas.width, y: Math.random() * this.canvas.height, size: Math.random() *
1.5 + 1, vx: (Math.random() - 0.5) * 0.2, vy: (Math.random() - 0.5) * 0.2, alpha: Math.random() *
0.2 + 0.05, colorHue: Math.random() > 0.5 ? 200 : 340 }); } } spawnBurst(x, y, tint) { const h = tint
=== "cyan" ? 200 : 340; for (let i = 0; i < 15; i++) { this.particles.push({ x: x, y: y, size:
Math.random() * 2 + 1, vx: (Math.random() - 0.5) * 3, vy: (Math.random() - 0.5) * 3, alpha: 0.8,
colorHue: h, decayRate: Math.random() * 0.025 + 0.02 }); } } renderLoop() { this.ctx.clearRect(0,
0, this.canvas.width, this.canvas.height); for (let i = this.particles.length - 1; i >= 0; i--) { const p =
this.particles[i]; p.x += p.vx; p.y += p.vy; if (p.decayRate) { p.alpha -= p.decayRate; if (p.alpha <=
0) { this.particles.splice(i, 1); continue; } } if (p.x < 0) p.x = this.canvas.width; if (p.x >
this.canvas.width) p.x = 0; if (p.y < 0) p.y = this.canvas.height; if (p.y > this.canvas.height) p.y =
0;
const dx = this.cursorX - p.x; const dy = this.cursorY - p.y; const dist = Math.sqrt(dx * dx + dy *
dy); if (dist < 140) { const pullForce = (140 - dist) / 2500; p.x -= dx * pullForce; p.y -= dy *
pullForce; } this.ctx.fillStyle = hsla({p.colorHue}, 40%, 60%, {p.alpha}); this.ctx.beginPath();
this.ctx.arc(p.x, p.y, p.size, 0, Math.PI * 2); this.ctx.fill(); } if (this.particles.length < 40) {
this.generate(1); } requestAnimationFrame(() => this.renderLoop()); } }
class SplineRoadway { constructor() { this.viewport =
document.getElementById("roadmapViewport"); this.basePath =
document.getElementById("roadBase"); this.progressPath =
document.getElementById("roadProgress"); this.vehicle =
document.getElementById("pulseCar"); this.nodeElements = []; this.coordinates = []; this.init(); }
init() { this.buildRoadwayPoints(); this.renderProgressTrack(); } buildRoadwayPoints() { const
chaptersCount = 20; const widthStep = 100; this.coordinates = []; let pathString = "M 20 70";
this.coordinates.push({ x: 20, y: 70 }); for (let i = 1; i < chaptersCount; i++) { const x = i *
widthStep + 20; const y = 70 + Math.sin(i * 1.3) * 40; pathString += L ${x} ${y};
this.coordinates.push({ x: x, y: y }); } this.basePath.setAttribute("d", pathString);
this.progressPath.setAttribute("d", pathString); } renderProgressTrack() {
this.viewport.querySelectorAll(".chapter-node").forEach(n => n.remove()); this.nodeElements =
[]; this.coordinates.forEach((pt, idx) => { const div = document.createElement("div");
div.className = "chapter-node"; div.style.left = ${pt.x}px; div.style.top = ${pt.y}px;
div.textContent = idx + 1; div.addEventListener("click", () => { if (idx <= state.unlockedChapter) {
setChapterIndex(idx); } else { audio.triggerFailure(); } }); this.viewport.appendChild(div);
this.nodeElements.push(div); }); } refreshState() { const count = 20; this.coordinates.forEach((pt,
idx) => { const node = this.nodeElements[idx]; node.className = "chapter-node"; if (idx <
state.unlockedChapter) { node.classList.add("completed"); } else if (idx ===
state.unlockedChapter) { node.classList.add("unlocked"); } if (idx === state.activeChapter) {
node.classList.add("current"); } }); const activePt = this.coordinates[state.activeChapter];
this.vehicle.style.left = ${activePt.x}px; this.vehicle.style.top = ${activePt.y}px;
const length = this.basePath.getTotalLength(); this.progressPath.style.strokeDasharray = length;
const fraction = state.unlockedChapter / (count - 1); const offset = length - (length * fraction);
this.progressPath.style.strokeDashoffset = offset;
const container = document.querySelector(".story-track-panel"); const centerOffset = activePt.x

```

```

- container.offsetWidth / 2; container.scrollTo({ left: centerOffset, behavior: "smooth" }); } } let
roadway;
function findEmptyPositions(board) { return board.map((cell, idx) => cell === "" ? idx :
null).filter(val => val !== null); }
function checkWin(board, marker) { const paths = [ [0, 1, 2], [3, 4, 5], [6, 7, 8], [0, 3, 6], [1, 4, 7],
[2, 5, 8], [0, 4, 8], [2, 4, 6] ]; return paths.some(set => set.every(idx => board[idx] === marker)); }
// Minimax search logic function minimaxAB(board, depth, alpha, beta, isMaxPlayer) { const
xWins = checkWin(board, "X"); const oWins = checkWin(board, "O"); const empty =
findEmptyPositions(board);
if (xWins) return 10 - depth; if (oWins) return -10 + depth; if (empty.length === 0) return 0;
if (isMaxPlayer) { let maxVal = -Infinity; for (let cell of empty) { board[cell] = "X"; let score =
minimaxAB(board, depth + 1, alpha, beta, false); board[cell] = ""; maxVal = Math.max(maxVal,
score); alpha = Math.max(alpha, score); if (beta <= alpha) break; } return maxVal; } else { let
minVal = Infinity; for (let cell of empty) { board[cell] = "O"; let score = minimaxAB(board, depth +
1, alpha, beta, true); board[cell] = ""; minVal = Math.min(minVal, score); beta = Math.min(beta,
score); if (beta <= alpha) break; } return minVal; } }
function computeAISelection(board, difficulty) { const empty = findEmptyPositions(board); if
(difficulty === "easy") { const randomElement = Math.floor(Math.random() * empty.length);
return empty[randomElement]; } if (difficulty === "medium") { if (Math.random() < 0.45) { const
randomElement = Math.floor(Math.random() * empty.length); return empty[randomElement]; } } if
(difficulty === "hard") { if (Math.random() < 0.15) { const randomElement =
Math.floor(Math.random() * empty.length); return empty[randomElement]; } } let bestVal =
Infinity; let move = null; for (let cell of empty) { board[cell] = "O"; const val = minimaxAB(board,
0, -Infinity, Infinity, true); board[cell] = ""; if (val < bestVal) { bestVal = val; move = cell; } } return
move; }
function systemStartup() { const storedProgress =
localStorage.getItem("protocol_progress_v2"); const storedScores =
localStorage.getItem("protocol_scores_v2"); if (storedProgress) { const decoded =
JSON.parse(storedProgress); state.unlockedChapter = decoded.unlocked; state.activeChapter
= decoded.current; } if (storedScores) { state.scores = JSON.parse(storedScores); }
refreshScoresHTML();
document.getElementById("modePvP").addEventListener("click", () =>
configureGameMode("pvp")); document.getElementById("modeEasy").addEventListener("click",
() => configureGameMode("easy"));
document.getElementById("modeImpossible").addEventListener("click", () =>
configureGameMode("impossible"));
document.getElementById("modeP2P").addEventListener("click", () =>
configureGameMode("p2p"));
document.getElementById("btnRestart").addEventListener("click", () => { clearActiveTimers();
resetInterface(true); if (state.mode === "p2p" && dataChannel && dataChannel.readyState ===
"open") { dataChannel.send(JSON.stringify({ type: "restart" })); } });
document.getElementById("btnPurge").addEventListener("click", () => { clearActiveTimers();
state.scores = { x: 0, o: 0, draws: 0 }; state.unlockedChapter = 0; state.activeChapter = 0;

```

```

localStorage.clear(); refreshScoresHTML(); configureGameMode("pvp");
roadway.refreshState(); audio.triggerFailure(); });
document.getElementById("btnP2PHost").addEventListener("click", () =>
setupP2PConnection(true));
document.getElementById("btnP2PClient").addEventListener("click", () =>
setupP2PConnection(false));
document.getElementById("btnCopyToken").addEventListener("click", copyP2PToken);
document.getElementById("btnProceedToken").addEventListener("click", processP2PToken);
document.querySelectorAll(".reaction-btn").forEach(btn => { btn.addEventListener("click", () => {
const emoji = btn.getAttribute("data-emoji"); sendReaction(emoji); }); });
document.querySelectorAll(".cell").forEach(node => { node.addEventListener("click", (e) =>
executeGridTurn(parseInt(node.getAttribute("data-idx")), e, true)); });
document.getElementById("audioToggle").addEventListener("click", () => { const muted =
!audio.muted; audio.setMuteState(muted); const path =
document.getElementById("audioWaves"); if (muted) { path.style.display = "none";
document.getElementById("audioToggle").style.color = "var(--text-muted)"; } else {
path.style.display = "block"; document.getElementById("audioToggle").style.color =
"var(--accent-x)"; audio.triggerConfirm(); } });
roadway = new SplineRoadway(); window.particles = new ParticleEngine();
configureGameMode("pvp"); }
function configureGameMode(targetMode) { clearActiveTimers(); state.mode = targetMode;
document.querySelectorAll(".menu-group button").forEach(btn =>
btn.classList.remove("active"));
document.getElementById("p2pPanel").classList.remove("visible");
document.getElementById("reactionBarContainer").classList.remove("visible");
if (targetMode === "pvp") { document.getElementById("modePvP").classList.add("active");
document.getElementById("activeModeText").textContent = "Local Link"; } else if (targetMode
=== "easy") { document.getElementById("modeEasy").classList.add("active");
document.getElementById("activeModeText").textContent = "AI Easy Node"; } else if
(targetMode === "impossible") {
document.getElementById("modeImpossible").classList.add("active");
document.getElementById("activeModeText").textContent = "AI Minimax Node"; } else if
(targetMode === "p2p") { document.getElementById("modeP2P").classList.add("active");
document.getElementById("activeModeText").textContent = "Bluetooth emulated link";
document.getElementById("p2pPanel").classList.add("visible"); resetP2PInterface(); } else if
(targetMode === "story") { document.getElementById("activeModeText").textContent = "Story
Array"; } resetInterface(false); refreshScoresHTML(); }
function setChapterIndex(idx) { clearActiveTimers(); state.activeChapter = idx; state.mode =
"story"; const details = gameChapters[idx];
document.getElementById("roadmapTitle").textContent = Chapter ${details.id}: ${details.name};
document.getElementById("chapterSummary").textContent = details.desc;
document.querySelectorAll(".menu-group button").forEach(btn =>
btn.classList.remove("active")); document.getElementById("activeModeText").textContent =

```

```

Sector ${idx + 1}; localStorage.setItem("protocol_progress_v2", JSON.stringify({ unlocked:
state.unlockedChapter, current: state.activeChapter })); roadway.refreshState();
resetInterface(false); }
function resetInterface(soundConfirmation) { state.board = Array(9).fill(""); state.currentPlayer =
"X"; state.running = true; document.getElementById("winningVector").style.display = "none";
document.getElementById("resetProgressBar").style.width = "0%";
document.querySelectorAll(".cell").forEach(cell => { cell.className = "cell"; });
if (state.mode === "p2p") { updateP2PStatusDisplay(); } else {
document.getElementById("systemStatus").textContent = "GRID STABLE. READY FOR
ACCESS."; }
if (soundConfirmation) { audio.triggerConfirm(); } }
function clearActiveTimers() { if (state.autoClearTimer) { clearTimeout(state.autoClearTimer);
state.autoClearTimer = null; } document.getElementById("resetProgressBar").style.width = "0%";
}
// WebRTC Direct Link Networking Stack function resetP2PInterface() { state.p2pRole = null;
state.p2pState = 'setup'; if (peerConnection) { peerConnection.close(); peerConnection = null; }
dataChannel = null; document.getElementById("p2pSetupSection").style.display = "block";
document.getElementById("p2pExchangeSection").style.display = "none";
document.getElementById("p2pRoleStatus").textContent = "Bluetooth Emulated Web-Link";
document.getElementById("reactionBarContainer").classList.remove("visible"); }
function setupP2PConnection(isHost) { audio.triggerConfirm(); state.p2pRole = isHost ? 'host' :
'client'; state.mySide = isHost ? 'X' : 'O';
document.getElementById("p2pSetupSection").style.display = "none";
document.getElementById("p2pExchangeSection").style.display = "block";
const configuration = { iceServers: [{ urls: "stun:stun.l.google.com:19302" }] }; peerConnection =
new RTCPeerConnection(configuration);
if (isHost) { state.p2pState = 'waiting-for-answer';
document.getElementById("p2pRoleStatus").textContent = "Role: Host Device (Side X)";
document.getElementById("p2pInstructions").textContent = "Step 1: Copy this Host Key and
send to Client device.";
dataChannel = peerConnection.createDataChannel("gameChannel");
setupDataChannelCallbacks(dataChannel);
peerConnection.createOffer().then(offer => { return peerConnection.setLocalDescription(offer);
}).then(() => { setTimeout(() => { const token =
btoa(JSON.stringify(peerConnection.localDescription));
document.getElementById("p2pTokenArea").value = token; }, 600); }); } else { state.p2pState =
'pasting-answer'; document.getElementById("p2pRoleStatus").textContent = "Role: Client
Device (Side O)"; document.getElementById("p2pInstructions").textContent = "Step 1: Paste
Host's key below and click 'Confirm Code'."; document.getElementById("p2pTokenArea").value
= ""; document.getElementById("p2pTokenArea").removeAttribute("readonly");
peerConnection.ondatachannel = (event) => { dataChannel = event.channel;
setupDataChannelCallbacks(dataChannel); }; } }
function setupDataChannelCallbacks(channel) { channel.onopen = () => { state.p2pState =

```

```

'connected'; document.getElementById("p2pExchangeSection").style.display = "none";
document.getElementById("p2pRoleStatus").textContent = "✅ Link Sync Active!";
document.getElementById("reactionBarContainer").classList.add("visible");
audio.triggerConfirm(); resetInterface(false); refreshScoresHTML(); }; channel.onclose = () => {
resetP2PInterface(); audio.triggerFailure(); refreshScoresHTML(); }; channel.onmessage =
(event) => { const payload = JSON.parse(event.data); if (payload.type === "move") {
executeGridTurn(payload.index, null, false); } else if (payload.type === "restart") {
resetInterface(false); } else if (payload.type === "reaction") {
showReactionPopup(payload.value); } }; }
function copyP2PToken() { const area = document.getElementById("p2pTokenArea");
area.select(); document.execCommand('copy'); audio.triggerConfirm(); const origText =
document.getElementById("btnCopyToken").textContent;
document.getElementById("btnCopyToken").textContent = "Copied!"; setTimeout(() => {
document.getElementById("btnCopyToken").textContent = origText; }, 1500); }
function processP2PToken() { const val =
document.getElementById("p2pTokenArea").value.trim(); if (!val) return; audio.triggerConfirm();
try { const sdpObj = JSON.parse(atob(val)); if (state.p2pRole === 'client' && state.p2pState ===
'pasting-answer') { peerConnection.setRemoteDescription(new
RTCSessionDescription(sdpObj)) .then(() => peerConnection.createAnswer()) .then(answer =>
peerConnection.setLocalDescription(answer)) .then(() => { setTimeout(() => { const
answerToken = btoa(JSON.stringify(peerConnection.localDescription));
document.getElementById("p2pTokenArea").value = answerToken;
document.getElementById("p2pTokenArea").setAttribute("readonly", "true");
document.getElementById("p2pInstructions").textContent = "Step 2: Copy this Client Code and
send it back to the Host."; state.p2pState = 'waiting-for-host-acknowledgement'; }, 600); }); } else
if (state.p2pRole === 'host' && state.p2pState === 'waiting-for-answer') {
peerConnection.setRemoteDescription(new RTCSessionDescription(sdpObj)) .then(() => {
document.getElementById("p2pRoleStatus").textContent = "Establishing local-link..."; }); } }
catch (err) { audio.triggerFailure(); alert("Invalid Link Token. Verify string contents."); } }
function updateP2PStatusDisplay() { if (state.p2pState !== 'connected') {
document.getElementById("systemStatus").textContent = "LINK OFFLINE. SETUP
BLUETOOTH PROTOCOL."; return; } if (state.currentPlayer === state.mySide) {
document.getElementById("systemStatus").textContent = YOUR TURN (${state.mySide}) -
TRANSMITTING; } else { document.getElementById("systemStatus").textContent =
OPPONENT TURN (${state.currentPlayer}) - WAITING; } }
// Reaction Broadcaster and Popup Engine function sendReaction(emoji) { if (state.mode ===
"p2p" && dataChannel && dataChannel.readyState === "open") {
dataChannel.send(JSON.stringify({ type: "reaction", value: emoji })); }
showReactionPopup(emoji); }
function showReactionPopup(emoji) { const board = document.getElementById("gameBoard");
const bubble = document.createElement("div"); bubble.className = "reaction-bubble";
bubble.textContent = emoji;
const xOffset = (Math.random() - 0.5) * 80; const yOffset = (Math.random() - 0.5) * 40;

```

```

bubble.style.left = calc(50% + ${xOffset}px); bubble.style.top = calc(50% + ${yOffset}px);
board.appendChild(bubble); audio.triggerClick();
setTimeout(() => { bubble.remove(); }, 1600); }
// Game Core Turns Action function executeGridTurn(idx, event, initiator) { if (!state.running ||
state.board[idx] !== "") { audio.triggerFailure(); return; }
if (state.mode === "p2p") { if (state.p2pState !== 'connected') { audio.triggerFailure(); return; } if
(initiator && state.currentPlayer !== state.mySide) { audio.triggerFailure(); return; } }
state.board[idx] = state.currentPlayer; const cell = document.querySelectorAll(".cell")[idx]; const
rect = cell.getBoundingClientRect(); const cx = rect.left + rect.width / 2; const cy = rect.top +
rect.height / 2; const burstHue = state.currentPlayer === "X" ? "cyan" : "magenta";
window.particles.spawnBurst(cx, cy, burstHue);
cell.classList.add(state.currentPlayer.toLowerCase()); audio.triggerClick();
if (state.mode === "p2p" && initiator && dataChannel && dataChannel.readyState === "open") {
dataChannel.send(JSON.stringify({ type: "move", index: idx })); }
if (evaluateTerminalMatch()) return;
if (state.mode === "pvp") { state.currentPlayer = state.currentPlayer === "X" ? "O" : "X";
document.getElementById("systemStatus").textContent = USER ${state.currentPlayer} SIGNAL
PENDING...; } else if (state.mode === "p2p") { state.currentPlayer = state.currentPlayer === "X"
? "O" : "X"; updateP2PStatusDisplay(); } else { state.currentPlayer = "O";
document.getElementById("systemStatus").textContent = "AI MATRIX RESOLVING CODES...";
state.running = false; setTimeout(() => { let selectionProfile = "impossible"; if (state.mode ===
"easy") selectionProfile = "easy"; if (state.mode === "story") { selectionProfile =
gameChapters[state.activeChapter].ai; } const move = computeAISelection(state.board,
selectionProfile); if (move !== null && move !== undefined) { state.board[move] = "O"; const
currentCell = document.querySelectorAll(".cell")[move]; const cRect =
currentCell.getBoundingClientRect(); const ccx = cRect.left + cRect.width / 2; const ccy =
cRect.top + cRect.height / 2; window.particles.spawnBurst(ccx, ccy, "magenta");
currentCell.classList.add("o"); audio.triggerClick(); } state.currentPlayer = "X"; state.running =
true; if (!evaluateTerminalMatch()) { document.getElementById("systemStatus").textContent =
"USER X SIGNAL PENDING..."; } }, 400); } }
function evaluateTerminalMatch() { const matchPatterns = [ [0, 1, 2], [3, 4, 5], [6, 7, 8], [0, 3, 6],
[1, 4, 7], [2, 5, 8], [0, 4, 8], [2, 4, 6] ]; for (let pattern of matchPatterns) { const [a, b, c] = pattern;
if (state.board[a] !== "" && state.board[a] === state.board[b] && state.board[a] ===
state.board[c]) { terminateSimulation(state.board[a], pattern); return true; } } if
(findEmptyPositions(state.board).length === 0) { terminateSimulation("draw", null); return true; }
return false; }
function terminateSimulation(result, vector) { state.running = false; clearActiveTimers();
let textMessage = ""; if (result === "draw") { state.scores.draws++; textMessage = "GRID
CONFLICT: EQUILIBRIUM STATE"; audio.triggerFailure(); } else { if (result === "X") {
state.scores.x++; textMessage = "ACCESS GRANTED: X DEFEATS O"; audio.triggerConfirm();
if (state.mode === "story" && state.activeChapter === state.unlockedChapter) { if
(state.unlockedChapter < 19) { state.unlockedChapter++; state.activeChapter =
state.unlockedChapter; setTimeout(() => { setChapterIndex(state.activeChapter); }, 1400); } else

```

```

{ textMessage = "COGNITIVE EVOLUTION ACHIEVED."; } } else { state.scores.o++;
textMessage = "ACCESS GRANTED: O DEFEATS X"; audio.triggerFailure(); }
renderWinningLine(vector); }
document.getElementById("systemStatus").textContent = ${textMessage}
(AUTO-CLEARING...);
const progress = document.getElementById("resetProgressBar"); progress.style.width =
"100%";
state.autoClearTimer = setTimeout(() => { resetInterface(false); }, 2500);
localStorage.setItem("protocol_scores_v2", JSON.stringify(state.scores));
refreshScoresHTML(); }
function renderWinningLine(pattern) { if (!pattern) return; const line =
document.getElementById("winningVector"); const cells = document.querySelectorAll(".cell");
const startCell = cells[pattern[0]]; const endCell = cells[pattern[2]]; const sRect =
startCell.getBoundingClientRect(); const eRect = endCell.getBoundingClientRect(); const bRect
= document.getElementById("gameBoard").getBoundingClientRect(); const startX = sRect.left -
bRect.left + sRect.width / 2; const startY = sRect.top - bRect.top + sRect.height / 2; const endX
= eRect.left - bRect.left + eRect.width / 2; const endY = eRect.top - bRect.top + eRect.height / 2;
const deltaX = endX - startX; const deltaY = endY - startY; const distance = Math.sqrt(deltaX *
deltaX + deltaY * deltaY); const rotationAngle = Math.atan2(deltaY, deltaX) * 180 / Math.PI;
line.style.width = ${distance}px; line.style.height = 4px; line.style.left = ${startX}px; line.style.top
= ${startY - 2}px; line.style.transform = rotate(${rotationAngle}deg); line.style.transformOrigin =
"0px 2px"; line.style.display = "block"; }
function refreshScoresHTML() { const valMy = document.getElementById("valMy"); const
valOpp = document.getElementById("valOpp"); const labelMy =
document.getElementById("labelMy"); const labelOpp = document.getElementById("labelOpp");
const scoreCardMy = document.getElementById("scoreCardMy"); const scoreCardOpp =
document.getElementById("scoreCardOpp");
let myVal = 0; let oppVal = 0;
if (state.mode === "p2p") { if (state.p2pRole === "host") { myVal = state.scores.x; oppVal =
state.scores.o; labelMy.textContent = "My Score (X)"; labelOpp.textContent = "Opponent (O)"; }
else if (state.p2pRole === "client") { myVal = state.scores.o; oppVal = state.scores.x;
labelMy.textContent = "My Score (O)"; labelOpp.textContent = "Opponent (X)"; } else { myVal =
0; oppVal = 0; labelMy.textContent = "My Score"; labelOpp.textContent = "Opponent"; } } else if
(state.mode === "pvp") { myVal = state.scores.x; oppVal = state.scores.o; labelMy.textContent =
"X Side"; labelOpp.textContent = "O Side"; } else { myVal = state.scores.x; oppVal =
state.scores.o; labelMy.textContent = "My Score (X)"; labelOpp.textContent = "AI Score (O)"; }
valMy.textContent = myVal; valOpp.textContent = oppVal;
scoreCardMy.classList.remove("scary-shake"); scoreCardOpp.classList.remove("scary-shake");
if (myVal < oppVal) { scoreCardMy.classList.add("scary-shake"); } else if (oppVal < myVal) {
scoreCardOpp.classList.add("scary-shake"); } }
window.addEventListener("DOMContentLoaded", () => { systemStartup(); }); </script>
</body></html>

```